

Combined Questions and Day 2-3 Review

Thirty-one verified problems across the current review set, eight question groups answered, and one consolidated explanation file. The earlier loop, mux, reset, and ripple-carry material is retained; three new operator and latch questions are added on pages 10-12.

31	8	1
verified problems covered	question groups answered	combined review PDF

Core mental model

Use an operator that matches the data width and the hardware intent: bitwise operators transform individual bits; logical operators reduce whole operands to truth values. In an incomplete combinational-style assignment, retaining a previous output value requires storage, so synthesis infers a latch.

- Nineteen additional saved **Last success** submissions were verified in the open HDLBits tabs.
- Three code-comment questions were found and answered together using the same peach tracker color.
- Problems with no code question are marked **No questions asked** in gray in the tracker.
- The earlier seven technical explanation pages remain unchanged in this combined PDF.

1. Verified problem set and question map

This map covers the previous twelve verified Day 2 problems and the nineteen additional successes from this update. Peach rows contain a question answered in this PDF; gray rows had no code question.

No.	HDLBits ID	Tracker problem	Question	Core point
19	Dff8r	DFF with reset	No question	Synchronous reset
20	Wire	Simple wire	No question	Direct connection
21	Bugs_nand3	NAND	No question	Port order + inversion
22	Fsm2	FSM 2 (async reset)	No question	OFF/ON Moore FSM
23	Vector100r	Vector reversal 2	Answered	genvar vs integer
24	Dff8p	DFF with reset value	No question	Negedge; reset 8'h34
25	Wire4	Four wires	No question	Named wire mapping
26	Popcount255	255-bit population count	Answered	initial, = vs ==
27	Fsm2s	FSM 2 (sync reset)	No question	Synchronous reset
28	Dff8ar	DFF async reset	No question	Reset in sensitivity list
29	Bugs_mux4	Mux	Answered	Selector hierarchy
30	Adder100i	100-bit adder 2	Explained	Ripple-carry generate
31	Notgate	Inverter	Answered	~ versus !
33	Dff16e	DFF with byte enable	No question	Per-byte enables
34	Bcdadd100	100-digit BCD adder	No question	100 generated stages
35	Andgate	AND gate	No question	Basic bitwise AND
36	Bugs_addsubz	Add/sub	Answered	& versus && in if
37	m2014_q4h	Wire	No question	Direct connection
38	m2014_q4a	D Latch	Answered	Latch inference
39	Fsm3onehot	One-hot transitions 3	No question	One-hot next state
40	Norgate	NOR gate	No question	Reduction NOR
41	m2014_q4i	GND	No question	Constant zero
42	m2014_q4b	DFF	No question	Async reset DFF
43	Fsm3	FSM 3 (async reset)	No question	Async reset FSM
44	Xnorgate	XNOR gate	No question	Bitwise XNOR
45	m2014_q4e	NOR	No question	Two-input NOR
46	m2014_q4c	DFF	No question	Sync reset DFF
47	Always_case	Case statement	No question	Combinational case
48	Fsm3s	FSM 3 (sync reset)	No question	Sync reset FSM
49	Wire_decl	Declaring wires	No question	Intermediate nets
50	m2014_q4f	Another gate	No question	Gate expression

Previously unanswered Day 1 questions already closed

Lemmings4	Why does the fall counter not need another reset?
Bugs_mux2	Why are bitwise and & correct, while logical and && are not?

2. genvar: what it is, and what it is not

Question

Why does genvar make sense in a generate-for loop, and why can a procedural loop use integer `i` instead?

Short answer

A genvar is an elaboration-only index. The elaborator substitutes each constant index and creates a separate structural item or module instance. The genvar does not become a changing signal in the circuit.

```
genvar i;
generate
  for (i = 0; i < 4; i = i + 1) begin : reverse_bits
    assign out[i] = in[3-i];
  end
endgenerate
```

The elaborated structure

```
assign out[0] = in[3];
assign out[1] = in[2];
assign out[2] = in[1];
assign out[3] = in[0];
```

Important refinement

Do not equate procedural for with a clock-time loop. A procedural loop inside always `@(*)` executes in the same simulation time step whenever the process runs. With constant bounds, synthesis normally unrolls it into hardware too. The distinction is structural elaboration semantics versus procedural block semantics.

Where	Module/generate scope	Inside always, initial, function, or task
Index	genvar	integer or another integral variable
When semantics apply	Elaboration	When the process executes
Natural use	Instances, named scopes, separate assigns	Algorithmic assignment to variables
Constant-bound synthesis	Creates repeated structure	Usually unrolled into repeated logic

3. Vector100r: both loop styles are valid

The accepted solution uses a procedural loop. This is valid because out is assigned inside a combinational always block. The loop visits all 100 positions in one process evaluation; it does not wait 100 clocks.

```
integer i;
always @(*) begin
    for (i = 0; i < 100; i = i + 1)
        out[99-i] = in[i];
end
```

Structural alternative

```
genvar i;
generate
    for (i = 0; i < 100; i = i + 1) begin : reverse_bit
        assign out[i] = in[99-i];
    end
endgenerate
```

Both descriptions reduce to fixed wiring. No storage or run-time selector is required. The two index forms describe the same reversal when i covers 0 through 99.

0	out[99] = in[0]	out[0] = in[99]	Two ends are paired
1	out[98] = in[1]	out[1] = in[98]	Next inward pair
98	out[1] = in[98]	out[98] = in[1]	Next inward pair
99	out[0] = in[99]	out[99] = in[0]	Two ends are paired

Choice rule

Use a procedural loop when one combinational or sequential process naturally owns the assignments. Use generate-for when stamping out instances, named scopes, or separate structural connections.

4. Adder100i: genvar stamps out a carry chain

The saved successful solution is a direct structural use of genvar: iteration *i* creates full-adder instance *i*. The carry array has 101 nodes because a 100-bit addition has one carry entering bit 0 and one carry leaving each of 100 stages.

```

wire [100:0] carry;
assign carry[0] = cin;

genvar i;
generate
  for (i = 0; i < 100; i = i + 1) begin : fa_stage
    fa dut (
      .a    (a[i]),
      .b    (b[i]),
      .cin  (carry[i]),
      .sum  (sum[i]),
      .cout (carry[i+1])
    );
  end
endgenerate

assign cout = carry[100:1];

```

0	a[0], b[0], carry[0]=cin	sum[0], carry[1]	Least-significant bit
i	a[i], b[i], carry[i]	sum[i], carry[i+1]	Generic generated stage
99	a[99], b[99], carry[99]	sum[99], carry[100]	Most-significant bit

Why cout = carry[100:1] works

HDLBits defines cout[i] as the carry output of bit *i*. Stage *i* writes carry[i+1], so carry[100:1] aligns exactly with cout[99:0].

- Keep carry[0] tied to cin; otherwise stage 0 has no defined carry-in.
- Use carry[i+1] for each stage output; carry[i] is that stage's input.
- Name the generate block so tools show readable hierarchy such as fa_stage[37].dut.
- The loop creates 100 full adders at elaboration; it does not reuse one full adder for 100 cycles.

5. Popcount255: initialization belongs inside combinational logic

Question

Why does an `initial` `begin` block not work for setting out to zero, and when should a procedural loop be used instead of `generate`?

Answer

An initial block runs once at simulation time zero. A population count must be recomputed every time the 255-bit input changes. Therefore out receives its default zero at the start of the combinational always block, before the loop adds the set bits.

```
always @(*) begin
    out = 8'd0;
    for (i = 0; i < 255; i = i + 1) begin
        if (in[i])
            out = out + 8'd1;
    end
end
```

out = 8'd0	Assignment	Reinitializes the accumulator on every evaluation
out == 8'd0	Comparison	Computes true/false and does not assign out
Blocking =	Immediate procedural update	Each iteration sees the previous iteration's sum
Nonblocking <=	Deferred update	Iterations tend to read the same old out
8-bit out	Range 0 through 255	Exactly enough for the maximum count

Procedural loop versus generate here

The procedural loop is natural because one combinational process owns one accumulator. Synthesis can build an adder network from the loop. A generate solution would need explicit partial sums or an adder tree; it must not create 255 drivers on one out vector.

Small cleanup

The test `if (in[i] && 1'b1)` is equivalent to `if (in[i])`. The shorter condition states the intent directly.

6. Mux questions: operators and selector hierarchy

Bugs_mux2: bitwise | versus logical ||

The mask implementation operates on all eight data bits, so bitwise operators are correct. Replicating sel produces an 8-bit mask. Logical operators convert each entire vector into one true/false result, destroying the per-bit data.

```
assign out = ({8{~sel}} & b) | ({8{sel}} & a);

// Equivalent and clearer:
assign out = sel ? a : b;
```

0	b	0	b
1	0	a	a

Bugs_mux4: how sel[0] and sel[1] divide the work

A 4-to-1 mux is a binary decision tree. The low-order selector bit chooses within each adjacent pair. The high-order selector bit chooses which pair reaches the output.

```
wire [7:0] lower_pair, upper_pair;
mux2 m0(sel[0], a, b, lower_pair);
mux2 m1(sel[0], c, d, upper_pair);
mux2 m2(sel[1], lower_pair, upper_pair, out);
```

00	pair outputs a and c	lower pair	a
01	pair outputs b and d	lower pair	b
10	pair outputs a and c	upper pair	c
11	pair outputs b and d	upper pair	d

Binary-address mental model

Read sel[1:0] as a two-bit address: sel[1] chooses the half of the input list, and sel[0] chooses the element inside that half.

7. Lemmings4: why the counter reset is already covered

Question

Why does count not need another explicit reset after a fall?

Answer

The accepted code already resets count on every path where the counter will be reused. On asynchronous reset, count becomes zero. During a fall it increments only while ground is low. On the landing edge, state is still a falling state and ground is high, so the sequential block schedules count ≤ 0 .

```

if (areset) begin
    state <= walkingLeft;
    count <= 5'd0;
end else begin
    state <= next_state;
    if (state == fallFromRight || state == fallFromLeft) begin
        if (!ground)
            count <= count + 5'd1;
        else
            count <= 5'd0;    // landing edge
    end
end
end

```

During fall	fallFromLeft/Right	Increment	Measures consecutive falling cycles
Landing edge	Still old falling state	Schedule zero	Prepares the next independent fall
After landing	Walking or splatter	Zero or irrelevant	Walking may fall again; splatter is terminal
areset	Any state	Immediate zero	Known startup state

Why the landing decision still sees the fall length

Nonblocking assignments update registers after the edge-triggered block evaluates. The landing comparison sees the old count for that edge; count is reset afterward. The design can decide survive versus splatter and reset the measurement on the same edge.

Counter design checklist

Define which edge counts as the first falling cycle, test the boundary around 20 cycles, saturate if useful, and reset at the event boundary where the measurement will be reused.

8. Other verified Day 2 takeaways

Dff8r	Synchronous reset is tested inside always @(posedge clk); q changes only on a rising edge.
Wire	A continuous assignment models a direct combinational connection with no storage.
Bugs_nand3	The provided module port order must match; NAND is the inverted AND result.
Fsm2	An asynchronous reset belongs in the sensitivity list; next-state logic remains combinational.
Dff8p	The DFF triggers on the falling edge and loads the exact reset value 8'h34.
Wire4	Each output is a named connection; duplicating b to x and y is intentional.
Fsm2s	A synchronous reset is checked after a clock edge; reset is not in the sensitivity list.
Dff8ar	An active-high asynchronous reset uses posedge areset alongside posedge clk.

One decision tree for future HDLBits loops

Creating repeated instances or structural scopes?	Yes	generate-for + genvar
Assigning or accumulating inside one always block?	Yes	procedural for + integer/integral index
Must work happen over multiple clock edges?	Yes	counter/FSM/register state, not a loop alone

Final recall

genvar is a generate variable used while elaborating repeated structure. A procedural loop index controls statements inside a process. Clock-to-clock repetition requires explicit registered state.

Problem references

<https://hdlbits.01xz.net/wiki/Vector100r>

<https://hdlbits.01xz.net/wiki/Popcount255>

https://hdlbits.01xz.net/wiki/Bugs_mux4

<https://hdlbits.01xz.net/wiki/Adder100i>

<https://hdlbits.01xz.net/wiki/Lemmings4>

https://hdlbits.01xz.net/wiki/Bugs_mux2

9. Inverter: ~ versus !

Question found in the successful Notgate code

Is ~ bitwise and ! logical?

Short answer

Yes. ~ complements every bit and preserves the operand width. ! asks whether the entire operand is logically false and returns one bit. For a single known 0/1 input they give the same inverter result, but they are not interchangeable for a vector.

```
// Best expression for the one-bit HDLBits inverter
assign out = ~in;

// Width contrast for a vector
~8'b0000_0010    // 8'b1111_1101
!8'b0000_0010    // 1'b0   (the vector is nonzero/true)
!8'b0000_0000    // 1'b1   (the vector is zero/false)
```

Why both look identical for this one-bit problem

The input and output are each one bit. For a known input of 0, both operators produce 1; for a known input of 1, both produce 0. The important difference is the rule they apply: ~ works bit by bit, while ! first converts the complete operand into one truth value.

Four-state note

Unknown or high-impedance bits can propagate an unknown result. That is another reason to choose the operator by intent instead of relying only on a small 0/1 example.

Recall

Use ~ for gates and per-bit inversion. Use ! when you need a one-bit true/false test of an entire expression.

<https://hdlbits.01xz.net/wiki/Notgate>

10. Add/sub: why & and && both work here

Question found in the successful Bugs_addsubz code

Why do `out & 8'b11111111` and `out && 8'b11111111` select the same branch in this `if`?

Short answer

The bitwise expression leaves `out` unchanged because the mask is all ones. The logical expression sees that the all-ones constant is true, so it reduces to the truth value of `out`. An `if` also converts its condition to a truth value. Therefore both test whether `out` is nonzero.

```
out & 8'hFF == out
out && 8'hFF == bool(out) && 1'b1
               == bool(out)

// Direct zero detector; reduction NOR is clear and width-safe.
result_is_zero = ~|out;
```

Step-by-step examples

out	out & 8'hFF	out && 8'hFF	if branch
8'h00	8'h00	1'b0	false
8'h02	8'h02	1'b1	true
8'hA0	8'hA0	1'b1	true

Do not generalize this equivalence

The two operators work the same only because the right-hand operand is an all-ones value and the result is immediately used as a condition. With `8'h0F`, bitwise `&` masks the upper four bits, while logical `&&` discards bit positions and returns only one bit.

Best expression for the intent

If the output should be high exactly when `out` is zero, write `result_is_zero = ~|out;` or `result_is_zero = (out == 8'b0);`. The intent is visible immediately.

https://hdlbits.01xz.net/wiki/Bugs_addsubz

11. D Latch: what 'latch inference' means

Question found in the successful Exams/m2014_q4a code

What does latch inference mean for these assignment lines, and why does the synthesis tool report a latch?

Short answer

When ena is 0, the code requires q to retain its previous value. Retaining a value requires storage. Because the control is a level signal rather than a clock edge, synthesis creates a level-sensitive D latch. The warning is expected here because the HDLBits problem intentionally asks for a latch.

```
// Intentional latch: no assignment while ena is low.
always @(*) begin
    if (ena)
        q <= d;
end

// This says the same thing, but q <= q is redundant:
// else q <= q;
```

How the inferred hardware behaves

ena	Latch behavior	Meaning
1	q follows d	The latch is transparent while enabled.
0	q holds its old value	The latch is closed and stores the last d value.

Why this matters in ordinary combinational logic

In a block meant to be purely combinational, every output must be assigned on every path. If one path leaves an output unchanged, a latch can be inferred accidentally. Here it is intentional; elsewhere it is often a bug. In SystemVerilog, a `always_latch` can state the intent more explicitly.

Latch versus flip-flop

A latch is **level-sensitive**: while enabled, output can respond to changes in d. A flip-flop is **edge-triggered**: output changes only at its specified clock edge.

Recall

No assignment (or `q <= q`) means 'remember the old value.' Remembering under a level enable is exactly the job of a latch.

https://hdlbits.01xz.net/wiki/Exams/m2014_q4a